

Packs outside of Templates

Document #: P2277R0
Date: 2021-01-03
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

§ 1 Introduction

There are several papers currently in-flight which introduce or rely on a facility that C++ does not currently have: the ability to use packs not just outside of variadic templates but also outside of templates entirely. Those papers are:

- [[P1061R1](#)] Structured Bindings can introduce a Pack
- [[P1858R2](#)] Generalized pack declaration and usage
- [[P1240R1](#)] Scalable Reflection in C++
- [[P2237R0](#)] Metaprogramming

The first two of these are papers in which I am an author and deal exclusively with being able to introduce and use packs in more places (as the titles might suggest). The latter two are broader papers about introducing reflection into C++, in which packs play a smaller role overall, though the question comes up.

Here is a short example which uses facilities from all four papers and demonstrates the issue:

```
1  template <typename... Ts>
2  struct simple_tuple {
3      // from P1858
4      Ts... elems;
5  };
6
7  int g(int);
8
9  void f(simple_tuple<int, int> xs) {
10     // from P1061
11     auto& [...a] = xs;
12     int sum_squares = (0 + ... + a * a);
13
14     // from P1858
15     int product = (1 * ... * g(xs.elems));
16
17     // from P1240, construct a reflection-range of the template
    parameters
18     // this should be a vector containing two reflections of 'int'
```

```

19     constexpr auto params =
      std::meta::parameters_of(reflexpr(decltype(xs)));
20
21     // from P1240: decltype(ys) is simple_tuple<int, int>
22     simple_tuple<typename(...params)> ys = xs;
23
24     // from P2237: decltype(zs) is simple_tuple<int, int>
25     simple_tuple<|params|...> zs = xs;
26 }

```

f here is *not* a template, yet we see five places where we’re using a pack in some way (on lines 11, 12, 15, 22, and 25).

While the two reflection-based usages look similar, there’s a significant distinction between the two. In [P1240R1], the `typename(...range)` transformation may look like a pack expansion, but it is not. We do not have access to each underlying element of the pack to be able to do further transformations on. But with [P2237R0], we do have a proper pack expansion, and can make such transformations we please. For example, if we wanted to make a `simple_tuple<int&, int&>` that refers to the underlying elements of `xs`, that involves taking the template parameters range (`params` from the above example) and adding an lvalue reference to each type. This is how we would do it with the two papers:

```

// with P1240
constexpr auto refs = params
    | std::views::transform(std::meta::add_lvalue_reference);
simple_tuple<typename(...refs)> ys_ref{a...};

// with P2237
simple_tuple<|params|&...> zs_ref{a...};

```

§ 2 Paper Response

[P1061R1] was last seen in Belfast, where Evolution approved the direction 12-5-2-0-1.

[P1858R2] was discussed on a telecon in October 2020. That paper is larger in scope and consists basically of three different sections, but the only relevant section as far as this paper is concerned is the question of exploring packs in more places (as in the example earlier). That section was received very favorably: 9-13-2-1-1. Other papers of the paper had less support (pack indexing slightly less at 7-11-6-2-0, and packifying much less so at 3-7-8-6-1), but the ability to simply have packs in more places is fundamental in that paper and already directly exposes having packs outside of templates.

In both cases, the source of opposition was implementation concern, along two axes:

- implementation effort (and to a lesser extent, specification effort)
- compilation latency

The first bullet point speaks for itself, but the second bears some elaboration. Right now, in order to handle packs properly, implementations have to cache expressions as they go. Since a `...` can appear arbitrarily late in an expression, and is only a suffix rather than a prefix, compilers just don’t know if

something will eventually become a pack expansion or not. This machinery only has to exist in *specifically* variadic templates today, since that’s the only place where we can get pack expansions today.

But if we added the facilities described in these papers, suddenly every function and even every declaration, could potentially have a pack expansion in it. Even at namespace scope!

```
simple_tuple<int, int> xs = {1, 2};  
simple_tuple<int*, int*> ptrs = {&xs.elems...};
```

Compilers would need to be able to handle all of this, which means they have to necessarily do more work. Even existing C++20 code that does not use any of these new language features could see its compile times increase.

[[P1240R1](#)] avoids this problem by only allowing very specific kinds of expansion, as demonstrated earlier. Notably, those specific kinds of expansions have identifiable prefixes, so that compilers can know in advance that a pack expansion is coming, and won’t have additional overhead for parsing regular functions (or arbitrary declarations) that don’t use the new features.

If we have a reflection-range of types, with [[P1240R1](#)] we can explode it into a template with those specific types as parameters. But that’s it. Any deviation from specific prescribes shapes requires more work from the user.

[[P2237R0](#)] does not have this restriction, and so runs into the same issue as my papers: implementation complexity and added compilation latency.

§ 3 What Do We Do About It?

I think there are three choices we can make about how to address this problem, but we have to make the decision holistically.

1. We allow packs outside of templates, understanding the necessary cost (both in time and latency) to make that work.
2. We allow packs outside of templates, but come up with a pack-expansion prefix or something to that effect. The prefixed expansion would be valid in both templates and non-templates, but only the suffixed expansion we already have would be valid in variadic templates. We also encourage people to start using prefixed expansion for compiler latency purposes.
3. We do not allow packs outside of variadic templates at all. If you want to use packs, you need to have a variadic template.

I think we should choose Option 1 (my preference here is probably not surprising, given that I’ve written two papers on extending pack usage). It’s useful functionality to have, and from a language perspective, it seems to make a lot of sense. Such examples arguably have very clear meaning.

Not all code is variadic templates. People do still, on occasion, I am told, write functions. Some problems lead themselves very well to pack expansion and we should give users all the tools available to solve their problems.

Barring that, Option 2 would be a little weird. One choice for a prefix annotation would be to use ellipsis (kind of makes sense in keeping with the theme), so we end up with like:

```
void some_func(int, int);

void f(simple_tuple<int, int> xs) {
    some_func(... xs.elems); // ok
    some_func(xs.elems ...); // error
}

template <typename... Ts>
void g(Ts... elems) {
    some_func(... elems); // ok
    some_func(elems ...); // ok
}
```

Same idea, two syntaxes? Delightful. Way better than one syntax, two ideas though. And as an idea it at least seems tolerable — at least the prefix syntax would be universally applicable.

Although option 2 might not necessary completely solve the issue anyway due to the existence of fold expressions. There, the ellipsis placement is driven by the form of expansion desired. $(E \text{ op } \dots)$ and $(\dots \text{ op } E)$ might mean different things, and the expression E could be arbitrarily involved. Does this imply the need for a prefix annotation indicating that a fold expression is coming? Not sure.

Option 3 is largely a decision to abandon [\[P1061R1\]](#) and most of [\[P1858R2\]](#). The goal of these papers is to avoid the sorts of complex workarounds necessary to write today. A less-strict version of option 3 might be to allow introducing packs into non-variadic templates but not outside of templates. That is, allow `g` below but not `f`:

```
void f(simple_tuple<int, int> xs) {
    auto [...elems] = xs; // error: pack outside of template
}

template <typename Tuple>
void g(Tuple ys) {
    auto [...elems] = ys; // ok: even if not a variadic template
}
```

But this leads to especially silly-looking workarounds, where we have to wrap code in generic lambdas... just because:

```
void h(simple_tuple<int, int> xs) {
    // we disallow this
    int product_bad = (1 * ... * xs.elems);

    // ... but we allow this
    int product_good = [](auto& xs){
        // need to disambiguate here, since now
        // xs.elems is dependent
        return (1 * ... * xs. ...elems);
    }(xs);
}
```

Which itself doesn't seem like a great place to end up. On the plus side, we would not need such workarounds in templates — only in non-templates.

To be clear, I don't think any of these options is an easy choice to just pick and go on a victory parade.

§ 4 Proposal

This paper isn't a proposal in of itself. We already have four different proposals in front of us solving four different problems.

But those proposals between them expose a common new language feature: the ability to have and use packs not just outside of variadic templates, but even outside of templates entirely.

The problem is: this is a difficult piece of functionality for implementations.

The question we have to answer is: Does the benefit of packs outside of templates (arguably fewer rules to think about and more straightforward code to write) outweigh the cost (large implementation effort and likely compile-time performance hit)? Or is the trade-off not there — we would rather invest implementation effort on *other* features even if it means that users will have to write the kinds of silly workarounds presented earlier?

While I have my personal preference (easy for me to claim, as not an implementor), I do think it would be valuable for Evolution to provide clear direction on this question earlier rather than later.

§ 5 Acknowledgements

Thanks to Daveed Vandevoorde for explaining the issues here to me, and then explaining them a few more times because I didn't understand them the first time.

§ 6 References

[P1061R1] Barry Revzin, Jonathan Wakely. 2019-10-07. Structured Bindings can introduce a Pack.
<https://wg21.link/p1061r1>

[P1240R1] Daveed Vandevoorde, Wyatt Childers, Andrew Sutton, Faisal Vali, Daveed Vandevoorde.
2019-10-08. Scalable Reflection in C++.
<https://wg21.link/p1240r1>

[P1858R2] Barry Revzin. 2020-03-01. Generalized pack declaration and usage.
<https://wg21.link/p1858r2>

[P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming.
<https://wg21.link/p2237r0>